

PANDAS

```
In [ ]: What is Pandas
Pandas is a fast, powerful, flexible and easy to use open source data ana
built on top of the Python programming language.
```

```
In [1]: import numpy as np
import pandas as pd
```

WORKING WITH SERIES

```
In [2]: #STRING
country = ['india','pakisthan','iran','iraq']
pd.Series(country)
```

```
Out[2]: 0      india
1    pakisthan
2        iran
3        iraq
dtype: object
```

```
In [3]: #integer
runs = ['20','45','37','87','87']
pd.Series(runs)
```

```
Out[3]: 0      20
1      45
2      37
3      87
4      87
dtype: object
```

```
In [5]: #custom index
marks = ['20','45','37','87','87']
subjects = ['ENGLISH','KANNADA','PYTHON','ELECTRONICS','PHYSICS']
pd.Series(marks,index=subjects)
```

```
Out[5]: ENGLISH      20
KANNADA      45
PYTHON      37
ELECTRONICS      87
PHYSICS      87
dtype: object
```

```
In [6]: #setting name
m1 = pd.Series(marks,index=subjects,name = 'marina ka markss')
m1
```

```
Out[6]: ENGLISH      20
KANNADA      45
PYTHON      37
ELECTRONICS      87
PHYSICS      87
Name: marina ka markss, dtype: object
```

```
In [ ]: SERIES FROM DICTIONARY
```

```
In [9]: dicti = {  
        "maths":67,  
        "science":98,  
        "social":78,  
        "python":97  
    }  
pd.Series(dicti)
```

```
Out[9]: maths      67  
        science   98  
        social    78  
        python    97  
        dtype: int64
```

```
In [11]: marks_series = pd.Series(dicti,name = "someone's marks")  
marks_series
```

```
Out[11]: maths      67  
        science   98  
        social    78  
        python    97  
        Name: someone's marks, dtype: int64
```

SERIES ATTRIBUTES

```
In [14]: #size returns the size  
marks_series.size
```

```
Out[14]: 4
```

```
In [15]: #dtype>>>> returns the datatype of given series  
marks_series.dtype
```

```
Out[15]: dtype('int64')
```

```
In [16]: #name ....returns the name mentioned while you create a series  
marks_series.name
```

```
Out[16]: "someone's marks"
```

```
In [17]: #unique>>>>> returns the unique valuee  
marks_series.unique
```

```
Out[17]: <bound method Series.unique of maths      67  
        science   98  
        social    78  
        python    97  
        Name: someone's marks, dtype: int64>
```

```
In [23]: runs = ['34','56','90','46','56','34','56']  
a = pd.Series(runs)
```

```
In [24]: a.unique
```

```
Out[24]: <bound method Series.unique of 0      34
1      56
2      90
3      46
4      56
5      34
6      56
dtype: object>
```

```
In [25]: #index.....returns the index values
marks_series.index
```

```
Out[25]: Index(['maths', 'science', 'social', 'python'], dtype='object')
```

```
In [26]: #values....returns the values mentioned in a string
marks_series.values
```

```
Out[26]: array([67, 98, 78, 97])
```

SERIES USING READ_CSV

```
In [29]: subs = pd.read_csv('subs.csv')
```

```
In [32]: movies = pd.read_csv('bollywood.csv')
```

```
In [35]: vk = pd.read_csv('kohli_ipl.csv')
```

```
In [36]: subs.head()
```

```
Out[36]:
```

	Subscribers gained
--	--------------------

0	48
1	57
2	40
3	43
4	44

```
In [37]: movies.head()
```

```
Out[37]:
```

	movie	lead
--	-------	------

0	Uri: The Surgical Strike	Vicky Kaushal
1	Battalion 609	Vicky Ahuja
2	The Accidental Prime Minister (film)	Anupam Kher
3	Why Cheat India	Emraan Hashmi
4	Evening Shadows	Mona Ambegaonkar

```
In [38]: movies.tail()
```

```
Out [38]:
```

	movie	lead
1495	Hum Tumhare Hain Sanam	Shah Rukh Khan
1496	Aankhen (2002 film)	Amitabh Bachchan
1497	Saathiya (film)	Vivek Oberoi
1498	Company (film)	Ajay Devgn
1499	Awara Paagal Deewana	Akshay Kumar

```
In [39]: vk.tail()
```

```
Out [39]:
```

	match_no	runs
210	211	0
211	212	20
212	213	73
213	214	25
214	215	7

```
In [41]: movies.sample(3)
```

```
Out [41]:
```

	movie	lead
488	Lucky Kabootar	Arvinder Bhatti
1160	Deadline: Sirf 24 Ghante	Rajit Kapoor
258	Irudhi Suttru	Madhavan

```
In [42]: movies.value_counts()
```

```
Out [42]:
```

movie	lead	
Gabbar Is Back	Akshay Kumar	2
Andaaz	Akshay Kumar	2
Tanu Weds Manu: Returns	Kangana Ranaut	2
Yuvvraaj	Salman Khan	1
Yuva	Ajay Devgn	1
..		
Yeh Mera India	Anupam Kher	1
Yeh Saali Aashiqui	Vardhan Puri	1
Youngistaan	Jackky Bhagnani	1
Yun Hota Toh Kya Hota	Paresh Rawal	1
Yakeen (2005 film)	Arjun Rampal	1

Name: count, Length: 1497, dtype: int64

```
In [49]: movies.sort_values(by = "movie", ascending = False, inplace = True)
```

```
In [50]: vk.sort_values(by='runs')
```

Out [50]:

	match_no	runs
7	8	0
86	87	0
92	93	0
90	91	0
205	206	0
...	...	...
163	164	100
119	120	100
122	123	108
125	126	109
127	128	113

215 rows × 2 columns

In [51]:

```
movies
```

Out [51]:

	movie	lead
941	Zor Lagaa Ke...Haiya!	Meghan Jadhav
758	Zokkomon	Darsheel Safary
672	Zindagi Tere Naam	Mithun Chakraborty
780	Zindagi Na Milegi Dobara	Hrithik Roshan
603	Zindagi 50-50	Veena Malik
...	...	...
484	2 States (2014 film)	Arjun Kapoor
1040	1971 (2007 film)	Manoj Bajpayee
725	1920: The Evil Returns	Vicky Ahuja
287	1920: London	Sharman Joshi
1022	1920 (film)	Rajniesh Duggall

1500 rows × 2 columns

SERIES MATHS METHOD

In [52]:

```
#count  
vk.count()
```

```
Out[52]: match_no    215
        runs        215
        dtype: int64
```

```
In [53]: #sum
        vk.sum()
```

```
Out[53]: match_no    23220
        runs        6634
        dtype: int64
```

```
In [54]: vk.prod()
```

```
Out[54]: match_no    0
        runs        0
        dtype: int64
```

```
In [55]: #mean
        subs.mean()
```

```
Out[55]: Subscribers gained    135.643836
        dtype: float64
```

```
In [56]: subs.median()
```

```
Out[56]: Subscribers gained    123.0
        dtype: float64
```

```
In [57]: subs.var()
```

```
Out[57]: Subscribers gained    3928.158513
        dtype: float64
```

```
In [58]: subs.std()
```

```
Out[58]: Subscribers gained    62.675023
        dtype: float64
```

```
In [59]: ## min/max
        movies.min()
```

```
Out[59]: movie    1920 (film)
        lead      Aadar Jain
        dtype: object
```

```
In [61]: vk.min()
```

```
Out[61]: match_no    1
        runs        0
        dtype: int64
```

```
In [62]: vk.max()
```

```
Out[62]: match_no    215
        runs        113
        dtype: int64
```

```
In [63]: #describe()
        vk.describe()
```

Out [63]:

	match_no	runs
count	215.000000	215.000000
mean	108.000000	30.855814
std	62.209324	26.229801
min	1.000000	0.000000
25%	54.500000	9.000000
50%	108.000000	24.000000
75%	161.500000	48.000000
max	215.000000	113.000000

SERIES INDEXING

In [64]: `vk[-1]` *#indexing does not workkkkk*

```

-----
-
KeyError                                Traceback (most recent call las
t)
File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\indexes\base.p
y:3805, in Index.get_loc(self, key)
    3804 try:
-> 3805     return self._engine.get_loc(casted_key)
    3806 except KeyError as err:

File index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas\_libs\hashtable_class_helper.pxi:7081, in pandas._libs.hasht
able.PyObjectHashTable.get_item()

File pandas\_libs\hashtable_class_helper.pxi:7089, in pandas._libs.hasht
able.PyObjectHashTable.get_item()

KeyError: -1

The above exception was the direct cause of the following exception:

KeyError                                Traceback (most recent call las
t)
Cell In[64], line 1
----> 1 vk[-1]

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\frame.py:4102,
in DataFrame.__getitem__(self, key)
    4100 if self.columns.nlevels > 1:
    4101     return self._getitem_multilevel(key)
-> 4102 indexer = self.columns.get_loc(key)
    4103 if is_integer(indexer):
    4104     indexer = [indexer]

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\indexes\base.p
y:3812, in Index.get_loc(self, key)
    3807 if isinstance(casted_key, slice) or (
    3808     isinstance(casted_key, abc.Iterable)
    3809     and any(isinstance(x, slice) for x in casted_key)
    3810 ):
    3811     raise InvalidIndexError(key)
-> 3812     raise KeyError(key) from err
    3813 except TypeError:
    3814     # If we have a listlike key, _check_indexing_error will raise
    3815     # InvalidIndexError. Otherwise we fall through and re-raise
    3816     # the TypeError.
    3817     self._check_indexing_error(key)

KeyError: -1

```

```
In [66]: vk[1:6]
```

Out [66]:

	match_no	runs
1	2	23
2	3	13
3	4	12
4	5	1
5	6	9

In [67]: vk[:, -1]

Out [67]:

	match_no	runs
214	215	7
213	214	25
212	213	73
211	212	20
210	211	0
...	...	...
4	5	1
3	4	12
2	3	13
1	2	23
0	1	1

215 rows × 2 columns

In [68]: vk[6:]

Out [68]:

	match_no	runs
6	7	34
7	8	0
8	9	21
9	10	3
10	11	10
...	...	...
210	211	0
211	212	20
212	213	73
213	214	25
214	215	7

209 rows × 2 columns

movies[4]# error

In [70]: movies[1:6]

Out [70]:

	movie	lead
758	Zokkomon	Darsheel Safary
672	Zindagi Tere Naam	Mithun Chakraborty
780	Zindagi Na Milegi Dobara	Hrithik Roshan
603	Zindagi 50-50	Veena Malik
1218	Zindaggi Rocks	Sushmita Sen

In [71]: movies[3:20:3]

Out [71]:

	movie	lead
780	Zindagi Na Milegi Dobara	Hrithik Roshan
1228	Zinda (film)	Sanjay Dutt
157	Zero (2018 film)	Shah Rukh Khan
1319	Zameer: The Fire Within	Ajay Devgn
1036	Yuvvraaj	Salman Khan
490	Youngistaan	Jackky Bhagnani

In [72]: movies[:, -1]

	movie	lead
1022	1920 (film)	Rajnesh Duggall
287	1920: London	Sharman Joshi
725	1920: The Evil Returns	Vicky Ahuja
1040	1971 (2007 film)	Manoj Bajpayee
484	2 States (2014 film)	Arjun Kapoor
...	...	...
603	Zindagi 50-50	Veena Malik
780	Zindagi Na Milegi Dobara	Hrithik Roshan
672	Zindagi Tere Naam	Mithun Chakraborty
758	Zokkomon	Darsheel Safary
941	Zor Lagaa Ke...Haiya!	Meghan Jadhav

1500 rows × 2 columns

```
In [74]: marks_series
```

```
Out[74]: maths      67
science    98
social     78
python     97
Name: someone's marks, dtype: int64
```

```
In [75]: marks_series[-1] # indexing works with dictionary in pandas
```

C:\Users\Admin\AppData\Local\Temp\ipykernel_11080\569294583.py:1: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`

```
marks_series[-1]
```

```
Out[75]: np.int64(97)
```

```
In [76]: marks_series[1:3]
```

```
Out[76]: science    98
social      78
Name: someone's marks, dtype: int64
```

```
In [77]: #fancyindexing
vk
```

Out[77]:

	match_no	runs
0	1	1
1	2	23
2	3	13
3	4	12
4	5	1
...	...	...
210	211	0
211	212	20
212	213	73
213	214	25
214	215	7

215 rows × 2 columns

In [82]: `vk.iloc[[1,2,3]]`

Out[82]:

	match_no	runs
1	2	23
2	3	13
3	4	12

In []: